

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE

Guion de Sustentación

Sistema de Chat en Tiempo Real

Integrantes:

César Tipán Antón
Erick Cordova
Camillie Ayovi
Mateo Aguilar

Docente:

Ing. Joel Alejandro Barba Salazar, Mg.

Asignatura:

Aplicaciones Distribuidas

Guayaquil, julio de 2026

Guion de sustentación

Duración prevista: 17 minutos de exposición y 5 de preguntas. Cada bloque lo presenta un integrante distinto, que defiende la parte que desarrolló.

Preparación previa

- Abrir el enlace público **antes** de empezar. El servidor gratuito se suspende por inactividad y la primera petición tarda cerca de un minuto.
- Tener dos navegadores listos con sesiones distintas: una ventana normal y una de incógnito. El token vive en `localStorage`, y dos pestañas normales comparten la misma sesión.
- Dejar abierta una tercera pestaña en `/docs`, la documentación interactiva de la API.
- Reducir el ancho de una ventana para mostrar el comportamiento en móvil.

Bloque 1 — Problema y arquitectura (4 min)

1. El problema: varios usuarios deben comunicarse en tiempo real a través de un servidor central, con autenticación y seguridad en los datos.
2. Presentar el diagrama de despliegue: cliente Angular en Vercel, servidor FastAPI en Render, PostgreSQL gestionado en Neon.
3. **Tecnologías y por qué.** Servidor: FastAPI, elegido por su soporte nativo de WebSocket sobre ASGI; SQLAlchemy en modo asíncrono con `asyncpg`, para no bloquear el bucle de eventos en cada consulta; `bcrypt` y JWT para credenciales y sesión. Cliente: Angular con señales, que redibujan solo el fragmento afectado cuando llega un evento por el socket. Pruebas: `pytest`, `Vitest` y `Playwright`.
4. Explicar la separación de los dos canales de comunicación: REST para operaciones puntuales sin estado, WebSocket para el flujo continuo del chat.
5. **Justificar la elección de WebSockets frente a sockets TCP crudos**, que es la decisión de diseño central del proyecto:
 - Las plataformas gratuitas de despliegue solo exponen puertos HTTP/HTTPS. Un socket TCP crudo no admite enlace público, y el enunciado lo exige.
 - WebSocket es una API de sockets sobre TCP: conserva la conexión persistente y bidireccional, y añade el handshake HTTP que permite atravesar proxies.
 - Con HTTP convencional habría hecho falta *polling*: más latencia y más carga, porque el servidor no puede iniciar la comunicación.

Bloque 2 — Servidor (4 min)

1. Recorrer la separación en capas: presentación, negocio, acceso a datos. Cada capa solo conoce la inmediatamente inferior.
2. **Autenticación.** Las contraseñas se guardan con `bcrypt` y un *salt* aleatorio por usuario: dos contraseñas iguales producen hashes distintos. El servidor emite un JWT firmado con HS256 y no guarda sesión alguna.
3. **Seguridad de los datos transmitidos.** TLS en los tres tramos (HTTPS, WSS y la conexión con la base de datos), CORS restringido a orígenes declarados, validación de entrada con `Pydantic` y consultas parametrizadas frente a la inyección SQL.
4. **Gestión de conexiones simultáneas.** Mostrar la estructura de `ConnectionManager`: cada usuario tiene un *conjunto* de sockets, no uno solo. Es lo que permite varias pestañas por cuenta sin duplicar al usuario en la lista.

Bloque 3 — Cliente y comunicación por sockets (4 min)

1. Rutas de la aplicación y protección de /chat mediante un *guard*.
2. Cómo viaja el token: cabecera Authorization en las peticiones HTTP; parámetro de consulta en la URL del WebSocket, porque el WebSocket del navegador no admite cabeceras propias.
3. Recorrer el protocolo de eventos con el diagrama de secuencia: history_batch al entrar, message, typing, user_joined, user_left y user_list.
4. Señalar por qué user_list es la única fuente de verdad de la lista de conectados: un cliente que se conecta tarde nunca vio las entradas anteriores, de modo que no puede reconstruir la lista a partir de user_joined.
5. Códigos de cierre propios: 4001 para token inválido, 4002 para UUID inválido del canal, 4004 para canal inexistente (UUID válido pero no existe en la BD).

Bloque 4 — Demostración en vivo (3 min)

Secuencia recomendada, en este orden:

1. Registrar una cuenta nueva delante del tribunal. Se entra al chat automáticamente.
2. En el segundo navegador, iniciar sesión con otra cuenta y entrar al mismo canal. **Señalar que ambos aparecen en la lista de conectados**, y que ninguno recargó la página.
3. Escribir un mensaje en un navegador y mostrar cómo llega al instante al otro. Detenerse un momento al escribir para que se vea el aviso de «está escribiendo».
4. Cambiar uno de los navegadores al canal tech y mostrar que desaparece de la lista de general: la presencia es por canal.
5. Recargar la página para demostrar que el historial persiste.
6. Cerrar el segundo navegador y mostrar cómo desaparece de la lista del primero.
7. Estrechar la ventana para mostrar el menú lateral deslizable.

Si el enlace público falla, continuar con la instancia local ya arrancada.

Bloque 5 — Despliegue, pruebas y cierre (2 min)

1. **Proceso de despliegue.** El despliegue es continuo: cada integración en main publica una versión nueva. El servidor se construye desde un Dockerfile de dos etapas, de modo que los compiladores no llegan a la imagen final, y Render comprueba que está vivo mediante la sonda /health. El cliente se compila y se publica como archivos estáticos en Vercel. Las credenciales viven en el panel de cada servicio, nunca en el repositorio.
2. 84 pruebas automatizadas: 46 unitarias del servidor, 29 del cliente y 9 de extremo a extremo sobre dos navegadores reales.
3. Explicar que se verificó que las pruebas fallan cuando se retira el código que verifican: una prueba que nunca falla no demuestra nada.
4. Cerrar con las limitaciones conocidas, admitidas de forma explícita: sin reconexión automática, sin gestión de la expiración del token en el cliente, y presencia limitada a un solo proceso del servidor.

Preguntas previsibles y respuestas

¿Esto usa sockets de verdad, o es HTTP disfrazado? WebSocket abre una conexión TCP persistente y full-duplex. El handshake se inicia por HTTP con la cabecera Upgrade, y a partir de ahí el protocolo deja de ser HTTP. El servidor envía datos sin que el cliente los pida, que es justamente lo que HTTP no permite.

¿Qué pasa si dos usuarios envían un mensaje a la vez? El servidor es asíncrono y de un solo hilo. Los mensajes se procesan uno tras otro en el bucle de eventos, de modo que no hay condiciones de carrera sobre la estructura de conexiones, y por eso mismo no se necesitan bloqueos.

¿Cómo se protege la contraseña? Nunca se almacena en claro. bcrypt aplica un *salt* aleatorio y un factor de coste, lo que hace inviables las tablas precalculadas y encarece la fuerza bruta. La verificación compara en tiempo constante.

¿Por qué el mensaje propio se dibuja cuando vuelve del servidor? Para mostrar exactamente lo que quedó persistido, con su identificador y su marca de tiempo definitivos. Si se dibujara al pulsar Enter, la pantalla podría mostrar un mensaje que en realidad falló al guardarse.

¿Qué ocurre si el usuario abre dos pestañas? Aparece una sola vez en la lista. Cada usuario mantiene un conjunto de sockets, y solo se le da por desconectado al cerrarse el último. Hay una prueba automatizada dedicada a este caso.

¿Y si el servidor lanza una excepción mientras alguien está conectado? La limpieza de la conexión vive en un bloque *finally*, de modo que el usuario se retira igualmente de la lista de conectados. De lo contrario quedarían usuarios fantasma: visibles como conectados, pero sin socket detrás.

¿Escala a miles de usuarios? No en su forma actual. La presencia se guarda en la memoria del proceso, así que con varias instancias del servidor cada una vería solo a sus propios conectados. Habría que compartir el estado mediante un canal de publicación y suscripción, por ejemplo Redis.

¿Por qué los identificadores son UUID y no enteros? Un entero secuencial revela cuántos usuarios o mensajes existen y permite enumerarlos. El UUID no aporta esa información.

¿Qué pasa con los mensajes de un usuario eliminado? Se conservan con el autor a nulo y se muestran como «Usuario eliminado». Los mensajes de un canal eliminado, en cambio, se borran en cascada, porque fuera de su canal no significan nada.